W5 Shared Code Guide

The notebooks and Sampling Explorer use the same reviewed implementation at **2026-fall-w5**. Each heading below opens the actual definition line in GitHub. The explanations are drawn from the implementation docstrings, so array shapes, side effects, and counting rules stay aligned. The notebook setup verifies the SHA-256 of all four downloaded modules.

Follow one calculation through the layers

UI controls → **static_view / filter_view** callbacks → **static_experiment / particle_filter** → numerical arrays → plot, diagnostic dictionary and CSV.

There is no fit/predict state hidden behind these functions. A call creates a new seeded experiment. The Gaussian benchmark uses the unchanged W4 model; exact sampling is an oracle for evaluation. MCMC draws include dependence, and particle weights and ancestry must be interpreted separately.

GaussianModel

Exact reference for one observed vector.

x and mean have shape (2,); noise, precision and covariance have shape (2,2); log_evidence is a scalar log density, not a probability. Construct with gaussian_model; treat stored arrays as read-only.

gaussian_model

Return an exact posterior/evidence reference, without fitting or sampling.

rho: observation-noise correlation, strictly between -1 and 1; it is NOT posterior correlation. x: observed length-2 vector, default (1,-1). Returned arrays are new. Raises ValueError for invalid dimensions/numbers. Example: gaussian_model(.8).mean is approximately [.833333,-.833333].

mean_field_optimum

Return the reverse-KL-optimal diagonal Gaussian (mean, variance).

Both outputs have shape (2,). The mean is the exact posterior mean; variance[j] = 1/precision[j,j], generally NOT covariance[j,j]. Inputs are unchanged; no iteration or download occurs.

Target

A two-dimensional benchmark with exact moments and an iid oracle.

kind is 'gaussian' or 'bimodal'; mean shape (2,), covariance (2,2). Construct with make_target. Bimodal means are (-3,0) and (3,0), equal weights, within-component covariance .25 I. Treat arrays as read-only. The iid oracle is available for this benchmark, not for general inference.

Target.log_density

Normalized log density at z of shape (...,2); return shape (...,).

A single length-2 vector returns a NumPy scalar. Natural logarithms. This benchmark knows the normalizer; MH needs only a proportional density. No samples or model state are changed by evaluation.

Target.sample

Return n independent oracle draws, shape (n,2), using local seed.

For the mixture each row chooses its own component. This is an exact benchmark sampler, not an MCMC transition or a weighted sample.

make_target

Build the W4 Gaussian posterior or a deliberately difficult mixture.

rho is the W4 OBSERVATION-noise correlation, not posterior correlation. gaussian_model validates -1 < rho < 1. The mixture ignores rho and has exact mean [0,0], covariance diag(9.25,.25). No fitting or random draws. Example: make_target().mean is [5/6,-5/6].

normalize_log_weights

Normalize a nonempty 1D log-weight array stably; return sum-one weights.

Minus infinity is allowed (zero weight); NaN, plus infinity and all-zero mass raise ValueError. A common finite offset changes no normalized weight. Example: log([1,2,7]) gives [.1,.2,.7]. No in-place mutation.

weight_ess

Return 1/sum(normalized_weights**2), a WEIGHT-concentration diagnostic.

Accept nonnegative finite 1D weights with positive sum; normalize internally. This is not MCMC autocorrelation ESS and cannot detect unsampled regions. Uniform N weights yield N; a single nonzero weight yields 1.

importance_sampling

Draw iid Gaussian proposals and return direct-IS/SNIS building blocks.

q has mean target.mean + [proposal_shift,0] and covariance proposal_scale**2 * target.covariance. Scale is a SD multiplier (>0), shift is in coordinate units. Return samples (n,2), normalized weights (n,), log_weights (n,), weight_ess, log_normalizer_estimate, and evaluation counts. Both benchmark targets have support R^2; this does not guarantee finite variance for a narrow q. No support-coverage test is inferred from ESS.

snis_summary

Summarize a scalar function on iid proposals with normalized IS weights.

values/weights are length n>=2. Return estimate, weighted posterior SD, and plug-in delta-method MCSE sqrt(n/(n-1)*sum(W^2*(f-estimate)^2)). MCSE is asymptotic, requires suitable finite moments and adequate coverage; a small reported value is not a certificate. Not for correlated MCMC draws.

random_walk_mh

Run Gaussian random-walk Metropolis; retain EVERY rejected state.

log_density maps a finite vector (d,) to a scalar log target up to a constant. initial is a finite vector with finite log density; scale is proposal SD in each coordinate. draws counts transitions, excludes the initial state, and includes warmup if requested by the caller. Return samples (draws,d), accepted bool (draws,), initial copy, target_evaluations draws+1. No adaptation, thinning or automatic convergence claim.

gaussian_gibbs

Sample exact Gaussian conditionals, coordinate 0 then coordinate 1.

draws counts complete sweeps. Conditional variance is 1/precision[j,j]; the mean uses the latest other coordinate. Return samples (draws,2), substeps (2*draws+1,2), *including initial, and conditional_draws=2*draws. This is stochastic Gibbs, not deterministic CAVI or marginal iid sampling.

chain_diagnostics

ArviZ 0.22 diagnostics for one scalar observable, shape (chains,draws).

Supply >=2 chains, >=20 POST-warmup draws each, without flattening chains. Return rank-normalized folded/split R-hat, bulk/tail/mean ESS, mean MCSE, pooled mean/SD and retained count. Diagnostics concern the supplied f(z), not every possible posterior quantity. Nonfinite diagnostic outputs become None and an explicit warning; they are never replaced with a passing value.

autocorrelation

Return the biased sample ACF at lags 0..max_lag for one 1D trace.

Used for visualization, not for computing ESS. Constant traces return NaN beyond lag zero to avoid pretending that a stuck chain mixes perfectly.

mh_transition_matrix

Exact finite-state MH transition matrix, including reject/self mass.

probabilities is a positive length-K target (normalized internally). proposal is a KxK row-stochastic matrix q[j|i]. Return P[i,j]. Reverse proposal probabilities enter the ratio; q[i,j]=0 permits no such move. This small exact oracle supports the detailed-balance hand calculation.

simulate_tracking

Generate fixed synthetic truth and observations, both shape (steps,).

steps>=1, finite a, positive noise SDs. Initial state has variance one; no transition is applied before observation y[0]. Local RNG, no file/network access. Keep this seed fixed while varying particle-filter seeds.

kalman_filter

Exact scalar Gaussian filtering reference for the stated initial model.

observations: finite 1D y of length T>=1. Return mean/variance/predicted_mean/ predicted_variance/gain arrays (T,), and log_evidence scalar. Variance is posterior uncertainty, not the Monte Carlo variance of an estimator. Uses data up through t only (filtering); no future-data smoothing.

systematic_resample

Return N ancestor indices by systematic resampling of N weights.

weights: finite nonnegative 1D, positive sum; rng is a local NumPy Generator. Draw one uniform offset in [0,1/N) and use N equally spaced positions. Output shape (N,), integer indices in [0,N). Consumes rng but does not mutate weights. Resampled particles are dependent copies, not new evidence.

particle_filter

Bootstrap SIS/PF with optional resampling AFTER each measurement update.

y: finite 1D length T. particles=N>=2. threshold in [0,1]; zero means SIS. Resample when pre-resampling weight ESS < threshold*N. Propagation uses the state transition, so incremental weights are observation likelihoods. Initial particles are N(0,1), weighted by y[0]. All means/variances and particle/weight arrays are recorded BEFORE resampling. Return arrays mean, variance, ess_before, ess_after, unique_roots_after, resampled of shape (T,); samples, weights, root_labels_before, resampling_indices of shape (T,N). Roots refer to initial particle IDs; distinct positions do not imply distinct ancestry. log_evidence estimates log likelihood and is not unbiased in log. A collapsed or invalid computation raises; no uniform-weight fallback.

static_experiment

Run a fixed-target experiment and return real estimates plus raw draws.

method: 'IID MC', 'SNIS', 'MH', or 'Gibbs'. kind: gaussian/bimodal. draws>=100 counts total proposals for IID/SNIS, POST-warmup draws PER CHAIN for MH/Gibbs (four chains, 300 warmup transitions each). proposal_scale is IS SD multiplier, proposal_shift shifts IS coordinate 0, mh_scale is MH proposal SD. initial_center offsets coordinate 0 of dispersed starts. seed is local; no learned state. Return target, samples (N,2), weights (N,), chains (4,draws,2) or None, and scalar/JSON-ready metrics. Evaluation counts include warmup; conditional draws and exact iid oracle calls are different operations. Wall time is measured locally and is not a universal ranking.

static_view

Gradio callback: same controls as static_experiment -> Figure, dict, CSV path.

Computes a NEW seeded experiment each time; writes one summary CSV in a fresh temporary folder for download. Plots include exact target contours, estimator/trace, weights/ACF and the two distinct expectation estimates. No server is launched. Returned Figure belongs to the caller.

filter_view

Gradio callback -> Figure, metric dict, timewise CSV path for tracking.

particles>=2, observation_sd>0, resampling threshold in [0,1], integer seed for PARTICLES. Data seed stays 20260929. Changing observation_sd rescales the same simulated observation noise and changes the assumed likelihood; changing particles/threshold/particle seed leaves data identical. CSV includes truth, observations, Kalman/PF estimates, ESS and ancestor count. Shows filtering moments before resampling; no future observations are used.

build_sampling_app

Build (do not launch) a two-tab Gradio Blocks Sampling Explorer.

Return a fresh Blocks object. Buttons map visible controls in the same order as static_view/filter_view arguments to plot/JSON/CSV outputs. CPU-only, no account keys, no user uploads. Launch in Colab with share=True; that temporary URL depends on the active runtime. Keep the notebook link.

Library APIs used by the notebooks

- NumPy Generator: local reproducible random streams.
- NumPy average: weighted averages, with the axis explicitly chosen.
- SciPy logsumexp: stable normalization in the log domain.
- ArviZ R-hat, ESS, and MCSE: preserve chain and draw dimensions.
- Gradio Blocks: compose controls and outputs; button events connect callbacks.
- pandas DataFrame: collect named experiment results and export CSV.

In Colab, use Python help(function) or inspect.getsource(function) to inspect the downloaded implementation without leaving the notebook. All module downloads are public; no credentials or manual file upload are required. A hash mismatch stops execution instead of silently switching versions.